Міністерство освіти і науки України

Національний технічний університет України

Київський політехнічний інститут імені Ігоря Сікорського

Лабораторна робота № 6

« Міграції схем за допомогою Prisma ORM »

Роботу виконали

Студент групи ІО-45

Соловійщук

Назар

Роботу перевірив:

Русінов В.В.

Київ 2026

ЛАБОРАТОРНА РОБОТА № 6

Тема: Міграції схем за допомогою Prisma ORM

Мета:

Опанувати роботу з Prisma ORM для керування схемою бази даних PostgreSQL.
Навчитися виконувати інтроспекцію існуючої бази даних для генерації моделей.
Вивчити механізм створення та застосування міграцій для оновлення структури таблиць.
Перевірити цілісність даних та коректність зв'язків за допомогою Prisma Studio.

Хід виконання роботи

Для початку роботи було ініціалізовано проект Node.js та встановлено необхідні залежності Prisma. За допомогою команди `npx prisma init` було створено базову структуру файлів. У файлі `.env` було налаштовано шлях до бази даних `DATABASE_URL`

Було виконано команду npx prisma db pull, яка проаналізувала існуючі таблиці в PostgreSQL (створені в Лабораторній роботі №5) та автоматично згенерувала відповідні моделі у файлі schema.prisma

Згідно з інструкціями, до схеми були внесені наступні зміни:
Додавання нової моделі: Створено модель Review для зберігання оцінок та коментарів до фільмів .
Налаштування зв'язків: Додано зв'язок @relation між моделями movies та Review (один фільм може мати багато відгуків).
Оновлення існуючих таблиць: У модель inventory додано логічне поле isAvailable зі значенням за замовчуванням true для обліку наявності дисків

Для кожної зміни було запущено процес створення міграції за допомогою команди npx prisma migrate dev. Це дозволило:
Автоматично згенерувати SQL-скрипти для оновлення бази даних у папці prisma/migrations/.
Безпечно змінити структуру таблиць у PostgreSQL без втрати існуючих даних.

Фрагмент фінальної схеми Prisma

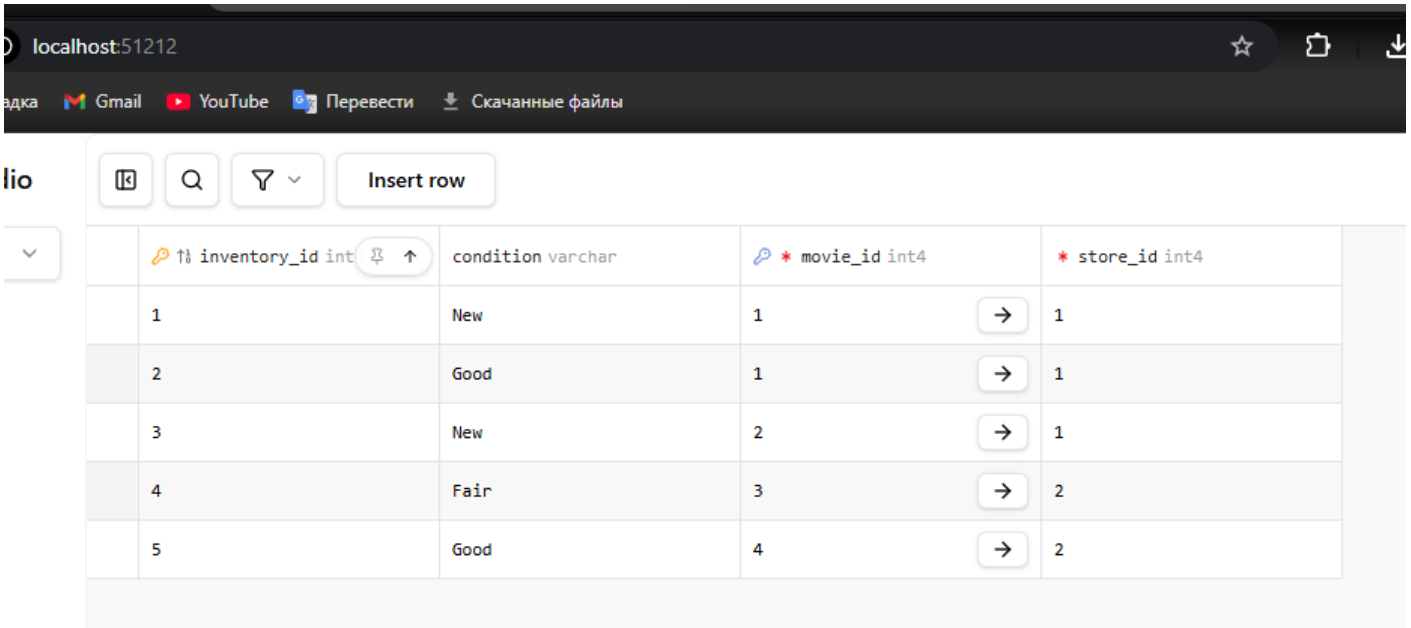
```
// Модель фільмів з новим зв'язком до відгуків
model movies {
  movie_id     Int       @id @default(autoincrement())
  title        String    @db.VarChar(255)
  inventory    inventory[]
  reviews      Review[]   // Зв'язок доданий під час міграції
  genres       genres    @relation(fields: [genre_id], references: [genre_id])
}

// Нова модель відгуків
model Review {
  id        Int      @id @default(autoincrement())
  rating    Int
```

```
  comment   String?
  movie     movies   @relation(fields: [movieId], references: [movie_id])
  movieId   Int
}
```

Для перевірки працездатності оновленої схеми було використано **Prisma Studio**. Через веб-інтерфейс було успішно додано новий запис у таблицю `Review`, що підтвердило коректність роботи зовнішніх ключів та типів даних.

```
  npm audit fix

Run `npm audit` for details.

C:\Users\klaus>npx prisma init --datasource-provider postgresql

Initialized Prisma in your project

  prisma/
    schema.prisma
  prisma.config.ts
  .env
  .gitignore

Next, choose how you want to set up your database:

CONNECT EXISTING DATABASE:
  1. Configure your DATABASE_URL in prisma.config.ts
  2. Run prisma db pull to introspect your database.

CREATE NEW DATABASE:
  Local: npx prisma dev (runs Postgres locally in your terminal)
  Cloud: npx create-db (creates a free Prisma Postgres database)

Then, define your models in prisma/schema.prisma and run prisma migrate dev to apply your schema.

Learn more: https://pris.ly/getting-started


C:\Users\klaus>
```

```
C:\Users\klaus>notepad .env

C:\Users\klaus>npx prisma db pull
Loaded Prisma config from prisma.config.ts.

Prisma schema loaded from prisma\schema.prisma.
Datasource "db": PostgreSQL database "movie_rental", schema "public" at "localhost:5432"

√ Introspected 6 models and wrote them into prisma\schema.prisma in 240ms

*** WARNING ***

These constraints are not supported by Prisma Client, because Prisma currently does not fully support check constraints.
 Read more: https://pris.ly/d/check-constraints
  - Model: "movies", constraint: "movies_duration_check"
  - Model: "payments", constraint: "payments_amount_check"

Run prisma generate to generate Prisma Client.


C:\Users\klaus>
```



inventory_id int	condition varchar	movie_id int4	store_id int4
1	New	1	1
2	Good	1	1
3	New	2	1
4	Fair	3	2
5	Good	4	2

Висновок:
У ході лабораторної роботи було вивчено сучасний підхід до керування базами даних за допомогою ORM. Використання міграцій Prisma дозволяє автоматизувати процес еволюції схеми даних, що значно полегшує розробку та підтримку проектів. Встановлено, що такий підхід є більш безпечним та структурованим у порівнянні з ручним написанням SQL-запитів для зміни структури таблиць.